

Integrity report- task 2

1. What did I find?

- Missing data (a total of 198 missing values)
- Duplicates
- Inconsistent values
- Invalid members_id

2. Where it was?

- **The missing data:**

```
the count of missing data before filling is: checkout_id      0
member_id            0
book_id              0
checkout_date        0
return_date          91
first_name           5
last_name            5
grade                41
neighborhood         5
membership_status    5
join_date            11
title                0
author               0
genre                0
pages                0
publication_year      35
publisher            0
```

- **Duplicates:**

I wanted the results to be true duplicates only and not similar so i made the subset to be checkout_id

```
Duplicated: [np.int64(9052), np.int64(9180), np.int64(9193),
np.int64(9194), np.int64(9246), np.int64(9280), np.int64(9296),
np.int64(9334)]
```

- **Inconsistent values:**

```
inconsistent values in neighborhood: ['Heliopolis' 'Zamalek' 'Nasr
City' 'Shubra' 'Maadi ' 'Maadi' 'zamalek'
'NASR CITY' 'HELIOPOLIS' nan]
inconsistent values in membership_status: ['Inactive' 'Active'
'inactive' 'active' nan]
```

- **Invalid members_id**

```
rows that have invalid member_id
checkout_id  member_id  book_id  checkout_date
395          9388      1104      515      2025-07-07
400          9393      1201      509      2025-07-10
402          9395      1104      526      2025-07-05
405          9398      1150      530      2025-07-10
415          9408      1201      523      2025-07-08
```

3. How big was it?

- **The missing data:**

There were in total 198 missing values.
Return's date was too large to drop without losing real checkouts
While grade and publication_year's smaller and i filled with median
since they are numeric

- **Duplicates:**

8 duplicate checkout_ids

- **Inconsistent values:**

neighborhood rows affected: 31
membership_status rows affected: 76

- **Invalid members_id:**

Rows with invalid member_id: 5

4.what i did and why?

- **The missing data:**

```
print("total:",df.isnull().sum().sum())
#i filled the numeric columns with median
```

```
df["grade"] = df["grade"].fillna(df["grade"].median())
df["publication_year"] =
df["publication_year"].fillna(df["publication_year"].median())
print("the count of missing data after filling
is:",df.isnull().sum())
```

Grade and publication_year are numeric columns so i filled with median (because median is not affected with outlier rather than mean)

Return_date and join_date are left null on purpose because it could mean that the book hasn't been returned yet. Filling them would be wrong info. And there are too many to drop.

First_name, last_name, neighborhood and membership_status were missing 5 rows. They are the same member_id that were invalid. I didn't fill them with the mode because it would be giving a fake identity. And these were later solved in problem 4.

- **Duplicates:**

I used

```
df = df.drop_duplicates(subset='checkout_id', keep='first')
```

I used a subset so that it would be true duplicates and kept the first.

Rows before removing duplicates: 417

Rows after removing duplicates: 409

- **Inconsistent values:**

I used

```
df['neighborhood'] = df['neighborhood'].str.strip().str.title()
df['membership_status'] =
df['membership_status'].str.strip().str.capitalize()
```

To remove whitespace and make the capitalization equal. I checked using .unique list before and after to make sure I didn't group different values together.

BEFORE:

```
inconsistent values in neighborhood: ['Heliopolis' 'Zamalek' 'Nasr
City' 'Shubra' 'Maadi']
```

```
inconsistent values in membership_status: ['Inactive' 'Active']
```

AFTER:

```
['Heliopolis' 'Zamalek' 'Nasr City' 'Shubra' 'Maadi']  
['Inactive' 'Active']
```

- **Invalid members_id:**

```
invalid = df[df['member_id'].isin(members_df['member_id']) == False]  
print("Rows with invalid member_id:", len(invalid))  
print(invalid[['checkout_id', 'member_id', 'book_id',  
               'checkout_date']])  
print("Rows before:", len(df))  
df = df[df['member_id'].isin(members_df['member_id']) == True]  
print("Rows after:", len(df))
```

I checked whether each checkout's member_id exists or not. i used == to pull out the invalid ones then i removed them while keeping the true ones only

Rows before: 409

Rows after: 404